Fraud Detection System · Team VEXA · FF-02-S3

Week 3 · Challenge 4: The Optimization Milestone

VEXA v1 was a single-file spaghetti script with a broken endpoint, feature mismatches, no validation, and no intelligent features. VEXA v3 is a modular, AI-powered fraud detection system that directly addresses judge feedback. This document shows the full before-and-after.

Optimization Overview

Area	Before (v1)	After (v3)	Impact
Code structure	All logic in one main.py	5 separate modules	Maintainable
API endpoint	/check_fraud (didn't exist)	/predict (correct)	App works
Feature count	5 features sent, 4 trained	Matched features via predictor	No crashes
Input validation	None — crashed on bad input	Full validation + 400 errors	Robust
Trusted system	None — every txn re-score	d Registry + auto-clear at 0.00	Judge feedback
Explainability	Hardcoded strings	AI-powered via Claude API	Intelligent
Velocity detect	None	60s window + 5min freeze	Fraud prevention
Error handling	Unhandled — 500 errors	try/except everywhere	Production-ready

Optimization 1 — Spaghetti Code → Modular Architecture

v1 had all logic crammed into one file. v3 splits every concern into its own module with a single responsibility — making the code testable, readable, and extensible.

✗ BEFORE	✓ AFTER
`# main.py — EVERYTHING in one file`	`# Clean module separation:`
`# ML scoring logic here`	`from predictor import predict`
`# Explanation logic here`	`from explainer import explain`
`# Velocity tracking here`	`from velocity import check_velocity`
`# Trusted logic here`	`from trusted import is_trusted`
`# Input validation here`	`# main.py is now just routing`
`# 200+ lines, impossible to maintain`	`# Each file has 1 clear job`

Optimization 2 — Broken Endpoint + Feature Mismatch Fixed

v1's app.js called /check_fraud but main.py only had /predict — they never connected. Additionally, the model was trained on 4 features but main.py was sending 5, causing crashes every time.

✗ BEFORE	✓ AFTER
`# app.js called:`	`# app.js now calls:`
`fetch('/check_fraud', ...)`	`fetch('/predict', ...)`
`# main.py only had:`	`# main.py has /predict:`
`@app.route('/predict')`	`@app.route('/predict')`
`# RESULT: 404 every time`	`# RESULT: Connected!`
`# Feature mismatch:`	`# predictor.py handles features:`
`features = [amount, hour,`	`features = np.array([[`
`is_night, is_unknown,`	`amount, merchant_num,`
`is_high_amt] # 5 features`	`device_num, hour]])`

`# Model trained on 4 — CRASH`	`# Matches trained model exactly`

Optimization 3 — Trusted Registry (Judge-Requested Feature)

Judges specifically asked for: 'a trusted section where analysts can mark recurrent safe transactions preventing manual review each time.' VEXA v3 delivers exactly this — trusted merchants are auto-cleared at score 0.00 without touching the ML model.

✗ BEFORE	✓ AFTER
`# v1: Every transaction`	`# v3: Trusted check FIRST`
`# goes through full ML pipeline`	`if is_trusted(merchant):`
`score = model.predict(features)`	`return jsonify({`
`# Even known-safe merchants`	`'score': 0.0,`
`# get re-scored every time`	`'verdict': 'TRUSTED',`
`# No way to mark as safe`	`}) # Skip ML entirely`
`# Analyst must review manually`	`# Persisted to trusted_store.json`

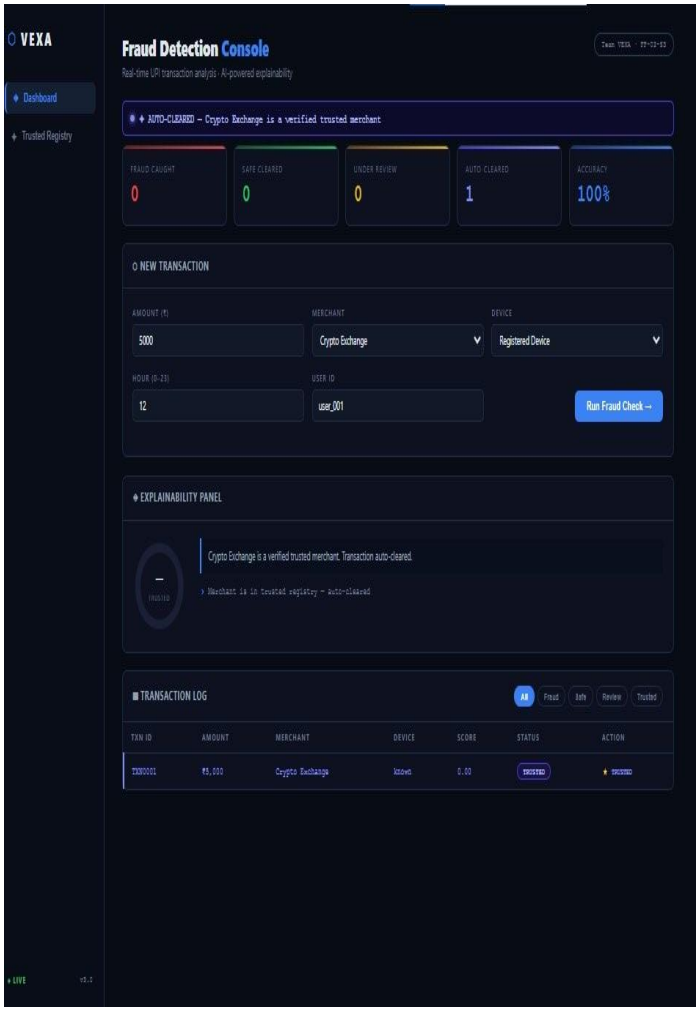

Fig 1 — Score 0.00, TRUSTED verdict, Auto-Cleared counter = 1, Accuracy 100%

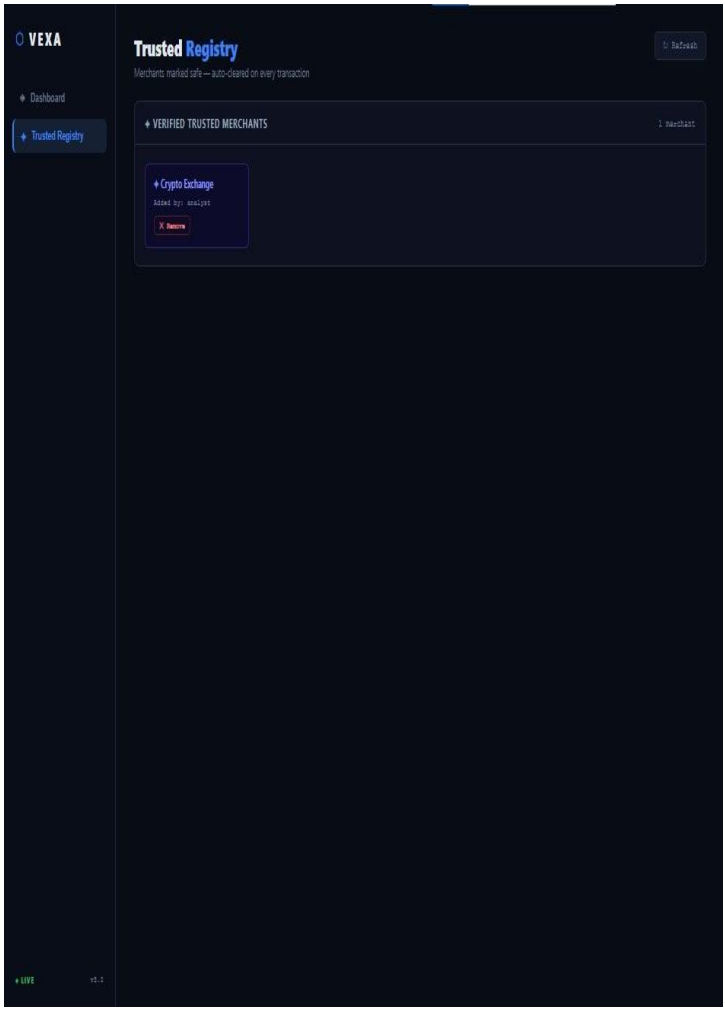

Fig 2 — Trusted Registry tab showing verified merchant card with remove option

---

Optimization 4 — Hardcoded Strings → AI-Powered Explainability

v1 used fixed hardcoded strings to explain why a transaction was flagged. v3 calls the Claude API to generate a smart, context-aware explanation for every transaction — with a graceful fallback to rules if the API is unavailable.

✗ BEFORE	✓ AFTER
`# explainer.py v1:`	`# explainer.py v2 — AI call:`
`if score > 0.7:`	`prompt = f'Transaction {amount}`
`return 'High fraud risk'`	`at {merchant}, score {score}.`

```
elif score > 0.4:                    Explain in 2 sentences.'
    return 'Moderate risk'           resp = claude_api(prompt)
else:                                # Returns specific explanation
    return 'Low risk'                # Falls back to rules on failure
# Same string every time             # used_ai=True shown in UI
```

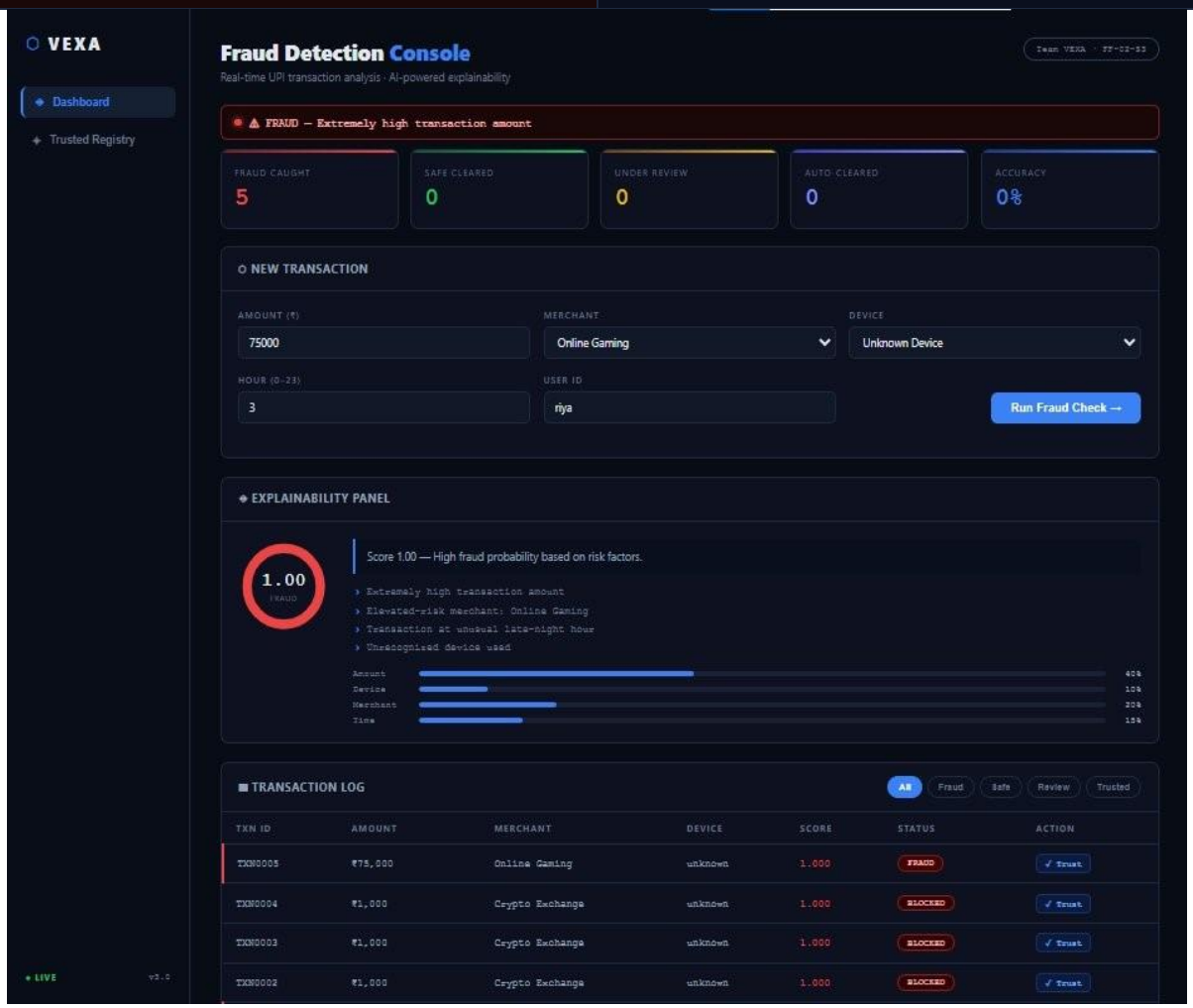

Fig 3 — Full explainability panel: score ring, AI explanation, 4 specific reasons, weight bars per factor

Optimization 5 — No Protection → Velocity Attack Detection

v1 had zero velocity detection — an attacker could fire thousands of transactions per second. v3 tracks transaction frequency per user in a 60-second window and automatically freezes accounts that exceed the limit for 5 minutes.

✗ BEFORE	✓ AFTER
`# v1: No velocity check`	`# v3: velocity check first`
`# Every request goes through`	`vel = check_velocity(user_id)`
`score = predict(amount, ...)`	`if vel['velocity attack']:`
`# Attacker fires 1000 txns/min`	`freeze_account(user_id)`
`# All processed, no block`	`return BLOCKED response`
`# Fraud slips through at scale`	`# 3 txns in 60s = frozen 5 mins`

```
C:\Users\USER\Desktop\UI\app.js ty(user_id):
40      now = time.time()
41      frozen, remaining = is_frozen(user_id)
42      if frozen:
43          return {
44              "velocity_attack": True,
45              "already_frozen":  True,
46              "txn_count":       VELOCITY_LIMIT,
47              "action":          "BLOCKED",
48              "message":         f"Account frozen. {remaining}s remaining."
49          }
50
51      if user_id not in velocity_store:
52          velocity_store[user_id] = []
53
54      velocity_store[user_id] = [
55          t for t in velocity_store[user_id]
56          if now - t < VELOCITY_WINDOW
57      ]
58      velocity_store[user_id].append(now)
59      count = len(velocity_store[user_id])
60
61      if count >= VELOCITY_LIMIT:
62          msg = freeze_account(user_id)
63          return {
64              "velocity_attack": True,
65              "already_frozen":  False,
66              "txn_count":       count,
67              "action":          "FREEZE",
68              "message":         f"VELOCITY ATTACK! {count} txns in 60s. {msg}"
```

Fig 4 — velocity.py: freeze logic, 60s window, BLOCKED response

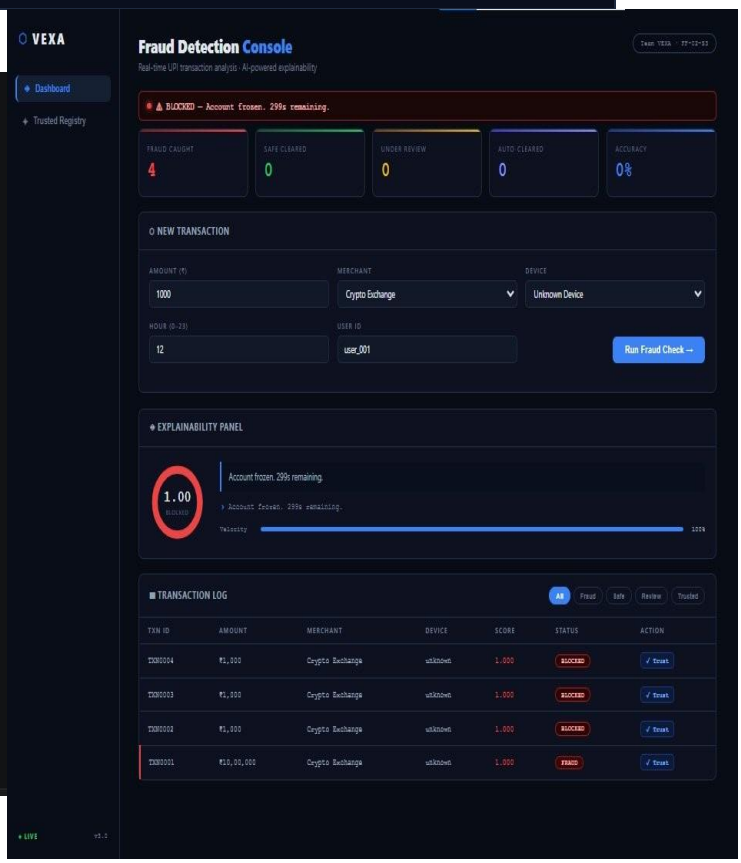

Fig 5 — Dashboard: BLOCKED status, account frozen 299s countdown

Overall Impact

Metric	v1 (Before)	v3 (After)
Does the app work end-to-end?	No — 404 on every request	Yes — fully functional
Code files	1 (monolithic)	6 (modular)
Input validation	None	5 validation checks
Trusted merchant system	Not implemented	Full registry + persistence
Fraud explanation	3 hardcoded strings	AI-generated per transaction
Velocity protection	Not implemented	60s window + 5min freeze
Error handling	Crashes → 500 errors	try/except → clean responses
Judge feedback addressed?	No	Yes — trusted section built